

COMS 4705 - Natural Language Processing

Midterm Exam - Fall 2022

Instructor: Daniel Bauer

Name: _____

UNI: _____

Total Points: 100

Time: 75 minutes

Instructions (please read):

The exam is closed books, closed notes. Calculators are permitted. Other electronic devices are not allowed.

The exam must be completed individually. During the exam, you are **not** allowed to communicate with anyone else taking the class. You must **not** disclose the content of the exam to anyone else.

Read the directions for each question carefully before answering.

Before submitting your exam, please initial each page.

Good luck!

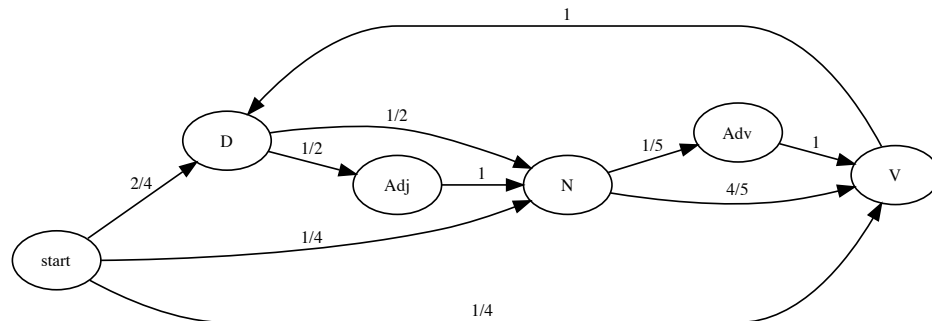
Academic Honesty Statement:

By submitting the exam you certify that you have neither given nor received unauthorized help on this exam and that you did not use any aids not specifically permitted. Any violation of this policy can result in an exam grade of zero and will be reported.

Signature: _____

Question 1 - Part-of-Speech Tagging with HMMs (20 pts)

Consider the following Hidden Markov Model (HMM).



Emission probabilities:

$$P(\text{*saw*}|V) = 5/6$$

$$P(\text{*well*}|V) = 1/6$$

$$P(\text{*saw*}|N) = 1/2$$

$$P(\text{*well*}|N) = 1/2$$

$$P(\text{*well*}|Adj) = 1$$

$$P(\text{*well*}|Adv) = 1$$

$$P(\text{*the*}|D) = 1$$

Use the **Viterbi algorithm** to compute the most likely sequence of POS tags for the phrase "*saw the well well*".

You can use the following table.

D	D	D	D
V	V	V	V
N	N	N	N
Adj	Adj	Adj	Adj
Adv	Adv	Adv	Adv
<i>saw</i>	<i>the</i>	<i>well</i>	<i>well</i>

The most probable POS sequence is:

Question 2 - Viterbi Algorithm revisited (18)

Recall the Viterbi algorithm that solves the decoding problem for HMMs. This algorithm computes the maximum joint probability of the token sequence w and any POS tag sequence (hidden state sequence). It uses a 2-dimensional table π .

Input: An input sequence w , consisting of tokens $w[1] \dots w[n]$

```
//initialization
Initialize  $\pi[0, \text{start}] = 1.0$ 
For each tag  $t$  in  $T$ , initialize  $\pi[0, t] = 0.0$ 

//main loop
for  $k = 1 \dots n$ :
    for each tag  $t$  in  $T$ :

         $\pi[k, t] = \max_s \pi[k-1, s] * P(t|s) * P(w[k] | t)$ 

return  $\max_s \pi[n, s]$ 
```

There are at most $|T|^n$ sequences of tags. However, not all tags t are possible for each time step k , either because the emission probability $P(w[k]|t) = 0$ or because some transition probabilities $P(t|s) = 0$.

Modify the Viterbi algorithm so that it computes the *number of possible tag sequences for the observed token sequence* that result in a non-0 probability.

You can cross out lines / make changes directly in the pseudocode above.

Question 3 - Parsing with PCFG (22 pts)

Consider the following Probabilistic Context Free Grammar (PCFG) in Chomsky Normal Form.

$S \rightarrow N VP$	$[1/4]$	$N \rightarrow \text{Mars}$	$[1/7]$
$S \rightarrow NP VP$	$[3/4]$	$N \rightarrow \text{news}$	$[2/7]$
$NP \rightarrow N N$	$[1/3]$	$N \rightarrow \text{reports}$	$[4/7]$
$NP \rightarrow NP PP$	$[2/3]$	$V \rightarrow \text{reports}$	$[1]$
$VP \rightarrow V PP$	$[1]$	$P \rightarrow \text{on}$	$[1]$
$PP \rightarrow P N$	$[1]$		

- (a) Show how the CKY algorithm would parse the sentence *"news reports on mars"* using this grammar. Fill out the following CKY table (you only need to record the nonterminals covering each span).

	0	1-news	2-reports	3-on	4-Mars
0					
1					
2					
3					
4					

(continues on next page)

(b) Show all parse trees for the sentence. For each parse tree, compute the probability for the tree according to the PCFG.

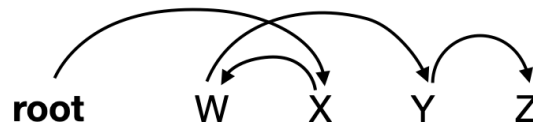
(c) What is the total probability of the sentence?

Question 4 - Transition-Based Dependency Parsing (20 pts)

Recall the **arc-standard transition-based dependency parser**. The three transitions are (as discussed in class):

- Shift: $(\sigma, w_i | \beta, A) \Rightarrow (\sigma | w_i, \beta, A)$
- Left-Arc: $(\sigma | w_i, w_j | \beta, A) \Rightarrow (\sigma, w_j | \beta, A \cup \{w_j, r, w_i\})$
- Right-Arc: $(\sigma | w_i, w_j | \beta, A) \Rightarrow (\sigma, w_i | \beta, A \cup \{w_i, r, w_j\})$

As mentioned in class, this transition system cannot generate non-projective dependency trees such as the following:



Assume now that we are adding the following additional operation:

- Swap: $(\sigma | w_i, w_j | \beta, A) \Rightarrow (\sigma | w_j, w_i | \beta, A)$

The Swap transition swaps the next word in the buffer with the word on top of the stack.

Demonstrate that, with this additional transition system, the non-projective dependency tree above can be generated. Show a sequence of transition steps, as well as the intermediate states. Start at state $(\mathbf{root}, [W, X, Y, Z], \emptyset)$

[extra space for question 4]

Question 5 - Linear Models and Feature Functions (20 pts)

In this problem we will use a **linear model** for Part-of-Speech (POS) tagging. The model predicts the POS tag t_i for each word w_i given the word itself, the previous word w_{i-1} , and the POS of the previous word t_{i-1} . To predict the POS for the word at position i , the model evaluates

$$t_i^* = \arg \max_{t_i} \Phi(w_{i-1}, w_i, t_{i-1}, t_i) \cdot \mathbf{v} = \arg \max_{t_i} \sum_j \Phi_j(w_{i-1}, w_i, t_{i-1}, t_i) \cdot v_j$$

where Φ is a feature function defined over the input words, previous tag, and a candidate output POS tag t_i , and $\mathbf{v}$ is a weight vector.

Consider the following training corpus:

- time/N flies/V like/P flies/N
- fruit/N flies/N like/V fruit/N
- flies/N time/V fruit/N

The function Φ should be an indicator function, which means it returns a vector of 0s and 1s.

- a) Define Φ by specifying the conditions under which each dimension of the feature vector is 1.
- b) Define a weight vector $\mathbf{v}$ so that the model assigns the correct POS to all tokens in the training corpus.

[extra space for question 5]